DD2525 Language-Based Security

Lab Assignment 3: Android Security Report

Group 28
May 12, 2026

Report Q&A

Task 1 - Intent Sniffing

Q1: Describe how the broadcast system works and the use cases of implicit broadcasts.

Android's broadcast system is a publish-subscribe mechanism where components publish events as Intent objects and any registered receiver matching the action receives the event. Implicit broadcasts have no specific target component - any receiver whose IntentFilter matches the action string will receive them. Common use cases include: system events (battery change, boot completed), intra-app communication between loosely coupled components, and inter-app data sharing (e.g., a location service broadcasting to multiple consumers).

Q2: Explain your code and how you can get these broadcast intents.

The IntentSniffer registers a BroadcastReceiver with registerReceiver() for both action strings used by ForegroundLocationService:

```
22        setContentView(R.layout.activity_main);
23
24        IntentFilter filter = new IntentFilter();
25        filter.addAction("tcs.lbs.locationapp.MainActivityReceiver"); // intra-app
26        filter.addAction("tcs.lbs.weather_app.WeatherBroadcastReceiver"); // inter-app
27
28        registerReceiver(locationReceiver, filter, Context.RECEIVER_EXPORTED);
```

Because sendBroadcast() in ForegroundLocationService specifies no receiverPermission, the Android system delivers the intent to all matching receivers system-wide, including IntentSniffer. On API 33+, RECEIVER_EXPORTED allows our receiver to receive cross-process broadcasts.

When the app registers interest in broadcast actions, the system delivers matching intents automatically. The sniffer extracts location: “ Location location = intent.getParcelableExtra("Location"); “ and then latitude and longitude are displayed on UI.

Q3: Propose and discuss TWO solutions to overcome intra-app broadcast sniffing.

Solution 1: LocalBroadcastManager / Internal Communication - Replacing global broadcasts with “LocalBroadcastManager” ensures message delivery within the same application process. This approach is simple to implement and makes them entirely inaccessible to external apps, but it is limited to the same app, internally.

Solution 2: Explicit Intents / Direct Component Communication - Instead of broadcasting internally, components can communicate directly through service binding, shared ViewModels, or direct method calls, which eliminates any possibility of external interception and provides strict control over data flow, but it does not scale to multiple listeners.

Q4: Can you explain why inter-app broadcasts can also be sniffed?

Inter-app sniffing is possible due to the open nature of implicit broadcasts. Since no target package is specified and no permissions are enforced in the current implementation, the action string “tcs.lbs.weather_app.WeatherBroadcastReceiver” is effectively public. Android's broadcast delivery mechanism routes implicit Intents to all installed applications with a matching receiver, meaning any third-party app including a malicious app, can register and receive sensitive location data without authorisation.

Q5: Propose and discuss TWO solutions for inter-app broadcast sniffing.

Solution 1 - Broadcast with custom signature permission: Declare a custom permission in LocationApp's manifest with protectionLevel="signature" and call sendBroadcast(weatherIntent, "tcs.lbs.RECEIVE_LOCATION"). Only apps signed with the same certificate and declaring <uses-permission> for this permission will receive the broadcast. IntentSniffer, signed differently, is denied. This preserves the broadcast architecture; only trusted apps can receive, but requires all legitimate receivers to share the same signing key.

Solution 2 - Explicit Intent to WeatherApp: Instead of a broadcast, send an explicit Intent to WeatherApp's specific BroadcastReceiver component using setComponent(). Explicit intents go directly to the named component and are not visible to other apps. This ensures no permission infrastructure is needed and completely eliminates the sniff. But the LocationApp must know WeatherApp's package name and component class at compile time.

Q6: Implemented Fixes with explicit intent (setPackage() methods):

```
locationAppIntent.setPackage(getPackageName());
weatherIntent.setPackage("tcs.lbs.weather_app");
```

Intra-app fix :

Here, getPackageName() refers to the package name of the LocationApp which restricts the broadcast, so that it can only be received inside the same application package. Even if another application registers the same action name, Android will not deliver the broadcast to

it because the package restriction only allows receivers belonging to the LocationApp. This prevents intra-app broadcasts from being exposed outside the application boundary.

Inter-app fix :

This restricts the broadcast delivery only to the WeatherApp package. The WeatherApp is the legitimate receiver of the location data, so Android forwards the broadcast only to that application. Any other application, including the IntentSniffer app, is blocked from receiving the broadcast even if it registers with the same IntentFilter. This prevents unauthorized inter-app broadcast sniffing.

Task 2 - Confused Deputy Attack

Q1: Describe the functionality of DataBaseActivity.

DataBaseActivity is a non-UI activity that acts as a database proxy for Notes. It is started by an Intent that contains two key extras: ACTION_NAME (one of SAVE_ITEM, DELETE_ITEM, GET_ITEM_TEXT, GET_ALL_ITEMS) and data extras such as NOTE_ID and NOTE_TEXT.

Based on ACTION_NAME, it calls the corresponding method on DataBaseHelper: insert(), update(), remove(), or getText(). Results are returned via setResult(RESULT_OK, data) and the activity finishes immediately.

Q2: What is the root cause of the issue?

The root cause of the vulnerability is that DataBaseActivity is declared as an exported component in the Android manifest - android:exported="true", which performs no verification of the caller's identity or permission. This means any app regardless of trust level can send an explicit Intent directly to “tcs.lbs.notes/.DataBaseActivity” with arbitrary ACTION_NAME and data extras. The activity blindly executes the requested database operation.

```
<activity
    android:name=".DataBaseActivity"
    android:exported="true"
    android:theme="@android:style/Theme.NoDisplay">
```

This results in a Confused Deputy vulnerability where the Notes app is tricked by an untrusted principal NotesExploit, into using its database access on behalf of the attacker.

Q3 & Q4: Propose two solutions with pros and cons.

Solution 1 - Remove export (implemented): Set android:exported="false" and remove the <intent-filter> from DataBaseActivity. When there is no intent-filter and no explicit exported=true, Android defaults to not exporting the component, so only components within the same app (tcs.lbs.notes package) can start it. This ensures Zero-overhead with no API

changes needed. But if a legitimate trusted partner app needed to invoke the activity, this solution would block it too.

Solution 2 - Signature-level permission: Add android:permission="tcs.lbs.notes.DB_ACCESS" to the DataBaseActivity declaration, and declare the permission with protectionLevel="signature" in the manifest. This allows trusted apps signed with the same key to still invoke the activity, offering flexibility. But it requires key management; if the signing key is compromised the protection fails which adds complexity to the build process.

Q5: Implemented Fix

In Notes/app/src/main/AndroidManifest.xml, the DataBaseActivity entry is changed to remove android:exported="true" and the <intent-filter>. The activity now reads:

```
<activity
    android:name=".DataBaseActivity"
    android:theme="@android:style/Theme.NoDisplay">
</activity>
```

NotesExploit's explicit intent to “tcs.lbs.notes.DataBaseActivity” now raises a SecurityException because the component is not exported. Verified by attempting each button in NotesExploit - all three result in a crash/permission denial instead of database modification.

NotesExploit Implementation

Three methods are implemented in NotesExploit_Template/MainActivity.java:

addClicked: Creates an explicit Intent with -

```
Intent intent = new Intent();
intent.setClassName(
        packageName: "tcs.lbs.notes",
        className: "tcs.lbs.notes.DataBaseActivity"
);

intent.putExtra( name: "ACTION_NAME",  value: "SAVE_ITEM");
intent.putExtra( name: "NOTE_ID",  value: "NEW_ID");
intent.putExtra( name: "NOTE_TEXT", noteText);
```

removeClicked: Creates an explicit intent with -

```
Intent intent = new Intent();
intent.setClassName(
        packageName: "tcs.lbs.notes",
        className: "tcs.lbs.notes.DataBaseActivity"
);

intent.putExtra( name: "ACTION_NAME",  value: "DELETE_ITEM");
intent.putExtra( name: "NOTE_ID", noteId);
```

showClicked: Creates an explicit intent with -

```
Intent intent = new Intent();
intent.setClassName(
        packageName: "tcs.lbs.notes",
        className: "tcs.lbs.notes.DataBaseActivity"
);

intent.putExtra( name: "ACTION_NAME",  value: "GET_ITEM_TEXT");
intent.putExtra( name: "NOTE_ID", noteId);
```

Task 3 - Deep-Link Injection

Q1: Root cause of the vulnerability.

The vulnerability has two components:

1. WalletActivity extracts the url query parameter from an incoming lbswallet://webview?url=<anything> deep link and passes it directly and without any validation to webView.loadUrl().

```
String encodedUrl = Uri.encode(suppliedUrl);
Uri deepLink = Uri.parse("lbswallet://webview?url=" + encodedUrl);
Intent intent = new Intent(Intent.ACTION_VIEW, deepLink);
```

2. The WebView has both setJavaScriptEnabled(true) and an unconditional webView.addJavascriptInterface(new WalletBridge(), "WalletBridge") in onCreate(), which exposes all @JavascriptInterface-annotated methods to any page loaded in the WebView.

```
webView.getSettings().setJavaScriptEnabled(true);
webView.addJavascriptInterface(new WalletBridge(), "WalletBridge");
```

Together, these mean that an attacker who controls the URL loaded in the WebView gains full access to balance-reading and money-transfer APIs.

Q2: Describe the exploitation method.

The attack proceeds as follows:

1. A malicious HTML page is hosted at http://127.0.0.1:8000/support.html (served by python -m http.server 8000, forwarded to the emulator via adb reverse tcp:8000 tcp:8000).

2. The page contains a link: <a href="lbswallet://webview?url=http://127.0.0.1:8000/support.html">Verify account</a>.

3. When the user taps the link in Chrome, Android resolves the lbswallet:// scheme and opens WalletActivity with the deep link intent.

4. WalletActivity.onCreate() extracts the url parameter (http://127.0.0.1:8000/support.html) and calls webView.loadUrl(url).

5. The WebView loads support.html - now in the WebView context, window.WalletBridge is available.

6. A window.addEventListener('load', ...) handler calls WalletBridge.getAllBalances() to read both user balances, then calls WalletBridge.transfer('user1', 'user2', '500') to transfer 500 SEK to the attacker's account.

Q3: Propose and discuss TWO solutions.

Solution 1 - URL allowlist (implemented): Before calling webView.loadUrl(), validate that the url parameter starts with file:///android_asset/. This restricts deep-link navigation to bundled app assets only, preventing any external URL injection. Pro: Minimal code change; preserves the deep-link functionality for asset-based support pages. Con: Does not mitigate risks if a bundled asset page is itself vulnerable; requires updating the allowlist if new asset paths are needed.

Solution 2 - Replace the url parameter with a content key: Instead of accepting arbitrary URLs, accept only a page identifier (e.g., lbswallet://webview?page=support) and map it to a hardcoded internal URL. Pro: No URL is ever parsed from an untrusted source; the set of valid pages is compile-time bounded. Con: Slightly more code; any new support page requires a manifest-style mapping update.

Q4: Implemented Fix

In WalletActivity.java, the URL extraction is changed to:

```java
String rawUrl = getIntent().getData().getQueryParameter( key: "url");

// FIX: Allowlist validation — only bundled asset URLs are accepted.
// The deep link lbswallet://webview?url=<anything> originally passed
// rawUrl directly to webView.loadUrl(), letting an attacker inject any
// arbitrary URL (including attacker-hosted pages) into the WebView that
// has WalletBridge attached.  Restricting to file:///android_asset/
// ensures only pages shipped with the app can be loaded via deep link,
// so WalletBridge methods are never exposed to external content.
if (rawUrl != null && rawUrl.startsWith("file:///android_asset/")) {
    deepLinkUrl = rawUrl;
}
// Non-conforming URLs are silently dropped; the app falls back to DEFAULT_URL.
```

URLs not matching the prefix are silently dropped; the app defaults to DEFAULT_URL (file:///android_asset/wallet_home.html). Verified: sending

lbswallet://webview?url=http://127.0.0.1:8000/support.html now opens the home page instead of the attacker's page, so WalletBridge is never exposed to external content.

Task 4 - Timing Attacks on Messengers

Implementation

ConversationActivity.java logs three events to Logcat with tag AppLog for each sent message:

new_msg: Timestamp when the message is sent (LocalTime.now()).

server_ack: Timestamp when the server's acknowledgment arrives.

receiver_ack: Timestamp when the receiver's delivery acknowledgment arrives.

Timing extraction command:

```
adb logcat -s AppLog > timing_log.txt
```

The Python script timing_analysis.py parses these entries and computes:

```
time_to_server   = server_ack.time - new_msg.time     # sender RTT to server
time_to_receiver = receiver_ack.time - server_ack.time # receiver RTT from server
```

Measured values are matched against Tables 1 and 2 from the lab spec using nearest-neighbour matching on (CDN_RTT + city_RTT).

Q1: How to infer user locations from timing data.

The server introduces artificial per-user delays based on their assigned city. The total one-way latency from user to server equals approximately CDN_RTT + city_RTT for that user. By measuring time_to_server we obtain the sender's combined latency; comparing against Table 1 identifies the region, then the residual against Table 2 identifies the city. Similarly, time_to_receiver reveals the receiver's location.

Example: if time_to_server ≈ 1187 ms, the closest match is Skyrim CDN (803) + Whiterun (375) = 1178 ms → sender is in Whiterun, Skyrim.

Q2: Difficulties in real-world messengers like WhatsApp or Signal.

- **No Logcat access:** Real apps do not expose per-message timestamps in accessible logs.
- **End-to-end encryption and padding:** Message contents and sizes are hidden; metadata is limited.
- **Variable network conditions:** Packet loss, retransmission, and congestion make delays highly variable and non-deterministic.
- **CDN edge nodes and load balancing:** Real CDNs route to the nearest edge, so measured latency reflects the edge node, not the origin server, and changes over time.

- **Message batching and buffering:** Messengers often coalesce messages and add artificial delays for battery efficiency, obscuring individual timing signals.
- **Tor/onion routing in Signal:** Signal supports Sealed Sender and is exploring onion routing; multiple hops make source inference impractical.

Q3: Possible fixes and evaluation.

- **Fix 1 - Server-side random delay injection:** The server adds a random uniform delay (e.g., 0–200 ms) to every acknowledgment before sending it. The random jitter makes the measured RTT unreliable for geo-inference. Pro: Cheap, server-side only. Con: Increases average message latency; attackers with many samples can average out the jitter.
- **Fix 2 - Message batching / fixed delivery windows:** Messages are held and delivered in fixed time windows (e.g., every 100 ms). All acknowledgments within a window are sent simultaneously. Pro: Flattens timing entirely within a window. Con: Increases maximum delivery latency; affects user experience in real-time chat.
- **Fix 3 - Onion routing / multi-hop relaying:** Route messages through multiple intermediate relay servers so no single observer can see both source latency and destination latency. Pro: Strong privacy guarantee. Con: Significantly increases infrastructure complexity and message latency.

Contributions

Jialei Qian: Task 1&3, report writing

Harshitha Samudrala: Task 2&4, report writing

AI Statement

Usage of AI-powered tools in this assignment meets the AI policy of the course as specified at the course Canvas page. AI tools such as Gemini and ChatGPT was used for explaining the course materials, error checking, and correcting the grammars of this report.